CIT300 - Data Structures and Algorithms
Individual Mid Assignment

Assignment Title: Mini Hospital Emergency Management System Using Data Structures

Task Overview

Design and implement a **Mini Hospital Emergency Management System** using appropriate data structures in **Java**.

The system should simulate the management of patients arriving at a hospital, including patient registration, emergency treatment requests, treatment completion, and patient visit history.

The following data structures **must** be implemented as part of the system.

1. Patient Records - Binary Search Tree (BST) - 20 Marks

Each patient should be stored in a **Binary Search Tree (BST)** using the **Patient ID** as the key.

The system should allow the user to:

- Insert a new patient
- Search for a patient using the Patient ID
- Delete a patient
- Perform an in-order traversal to display patients in ascending order of Patient ID

Each patient record should contain suitable information such as:

- Patient ID
- Patient Name
- Age
- Contact Number
- Medical Condition

2. Emergency Patient Queue - Queue - 20 Marks

Patients arriving at the emergency unit should be managed using a **Queue**.
The system should implement:

- **Enqueue** - add a patient to the waiting queue
- **Dequeue** - remove the next patient for treatment
- Display all patients currently waiting
- Appropriate handling of an empty queue

The queue should follow the **FIFO (First-In, First-Out)** principle.

3. Treatment History - Stack - 20 Marks

When a patient's treatment is completed, the treatment record should be stored using a **Stack**.

The system should implement:

- **Push** - add a completed treatment record
- **Pop** - remove the most recently completed treatment record
- Display treatment records
- Appropriate handling of an empty stack

The stack should follow the **LIFO (Last-In, First-Out)** principle.

4. Patient Visit History - Singly Linked List - 20 Marks

Each patient should have a **Singly Linked List** containing their previous hospital visits.

The system should allow:

- Adding a new visit to the patient's history
- Removing a visit
- Searching for a visit
- Displaying the patient's visit history

Each visit may contain information such as:

- Visit ID
- Visit Date
- Doctor Name
- Diagnosis
- Treatment

5. GitHub Repository & Development Evidence - 10 Marks

Each student must create and maintain a **GitHub repository** for this assignment.

The repository must contain:

- Complete Java source code
- Appropriate project structure
- README file
- Development history/commit history

Students are expected to commit their work **progressively throughout the development of the assignment**.

Examples of meaningful commits include:

- `Created project structure`
- `Implemented patient BST`
- `Added BST search and deletion`
- `Implemented emergency queue`
- `Implemented treatment stack`
- `Implemented patient linked list`

- `Added testing`
- `Updated README`

GitHub Requirements

- Each student must use their **own GitHub account**.
- The repository link must be submitted through the LMS.
- The repository should contain multiple meaningful commits demonstrating the development process.
- Students should **not upload the complete project as a single final commit**.
- The repository should remain accessible until the assignment has been fully assessed.

The GitHub repository and commit history may be reviewed as evidence of the student's development process.

6. Development & Demonstration Video - 10 Marks

Each student must record a **5-10 minute video** demonstrating their completed assignment. The video must include:

1. A brief introduction. The student's face must be visible during the introduction.
2. A brief explanation of the developed system.
3. The student's GitHub repository and commit history.
4. An explanation of how each data structure is used.
5. Demonstration of the system running.
6. Demonstration of important operations performed using:
    - BST
    - Queue
    - Stack
    - Singly Linked List
7. Explanation of important implementation/design decisions.
8. A brief reflection on what was learned from the assignment.

Final Submission Requirements

Each student must submit the following through the **LMS**:

1. **Java source code**
2. **GitHub repository link**
3. **Development and demonstration video**
4. **README file**
5. **Screenshots of program output**

Google Drive Submission

If the submission files are too large to upload directly to the LMS, students may upload the files to **Google Drive** and submit the shareable Google Drive link through the LMS.

The Google Drive folder/link must provide **Edit access** to:

- **asanka.r@sltc.ac.lk**
- **kaushika.w@sltc.ac.lk**

Students are responsible for checking that the submitted link works and that the above email addresses have the required access.

Email submissions will not be accepted, under any circumstances.

Assignment Marking Scheme

Component	Marks
Patient Records - Binary Search Tree	20
Emergency Patient Queue - Queue	20
Treatment History - Stack	20
Patient Visit History - Singly Linked List	20
GitHub Repository & Development Evidence	10
Development & Demonstration Video	10
Total	**100**

This assignment contributes 10% to the final grade.

Important Academic Integrity & Individual Work Requirement

This is an **individual assignment**. Each student is expected to design, implement, test, and explain their own solution.

Students may use learning resources and programming documentation to support their understanding. However, the submitted implementation must represent the student's own work.

The **source code, GitHub repository, commit history, screenshots, and video** may be reviewed to verify the authenticity and development of the submitted work.

Submitting another person's work or submitting a solution without understanding its implementation may result in appropriate academic action in accordance with university regulations.